

NBA Player Longevity Prediction — Feature Engineering Pipeline

Goal: Predict whether an NBA player will remain active for at least 5 years (`target_5yrs = 1`) using per-game statistics from their early career.

Dataset: 1,340 NBA players × 22 columns (20 statistical features + player name + target).

This notebook walks through every stage of the feature engineering pipeline:

1. Load data & define target variable
2. Drop non-predictive / leakage-risk columns
3. Correlation analysis & redundancy removal
4. Engineer composite efficiency features
5. Handle missing/edge-case values
6. Export model-ready dataset

1 Import Libraries & Load Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', '{:.3f}'.format)

df_raw = pd.read_csv('nba-players.csv')
print(f"Dataset shape: {df_raw.shape}")
df_raw.head()
```

Dataset shape: (1340, 22)

Unnamed: 0		name	gp	min	pts	fgm	fga	fg	3p_made	3pa	
0	0	Brandon Ingram	36	27.400	7.400	2.600	7.600	34.700	0.500	2.100	25.000
1	1	Andrew Harrison	35	26.900	7.200	2.000	6.700	29.600	0.700	2.800	23.500
2	2	JaKarr Sampson	74	15.300	5.200	2.000	4.700	42.200	0.400	1.700	24.400
3	3	Malik Sealy	58	11.600	5.700	2.300	5.500	42.600	0.100	0.500	22.600
4	4	Matt Geiger	48	11.500	4.500	1.600	3.000	52.400	0.000	0.100	0.000

✓ 2 Define the Target Variable

`target_5yrs` is our **binary dependent variable**:

Value	Meaning
1	Player remained active for ≥ 5 seasons
0	Player did not reach 5 seasons

We isolate it now so it is never accidentally transformed alongside the features.

```
# Isolate the target before any feature work
target = df_raw['target_5yrs'].copy()

print("Target distribution:")
print(target.value_counts())
print(f"\nClass balance – survived 5 yrs: {target.mean():.1%}")
```

```
Target distribution:
target_5yrs
1      831
0      509
Name: count, dtype: int64
```

```
Class balance – survived 5 yrs: 62.0%
```

✓ 3 Drop Non-Predictive Columns

Columns removed and rationale:

Column	Reason
Unnamed: 0	Row index artefact from CSV export — carries no information
name	Player name is an identifier, not a statistic. Including it would cause data leakage for well-known players

The target column `target_5yrs` is also removed from the feature matrix (X) and kept separately.

```
# Work on a copy so the raw data is always intact
df = df_raw.drop(columns=['Unnamed: 0', 'name', 'target_5yrs'])
```

```
print("Feature columns remaining:", df.columns.tolist())
print("Feature matrix shape:", df.shape)
```

```
Feature columns remaining: ['gp', 'min', 'pts', 'fgm', 'fga', 'fg', '3p_made', '3p_atm', 'ftm', 'fta', 'ft', 'oreb', 'dreb', 'trb', 'ast', 'stl', 'blk', 'tov', 'pf', 'pts_pct', 'fg_pct', '3p_pct', 'ft_pct', 'oreb_pct', 'dreb_pct', 'trb_pct', 'ast_pct', 'stl_pct', 'blk_pct', 'tov_pct', 'pf_pct', 'pts_per_game', 'fg_per_game', '3p_per_game', 'ft_per_game', 'oreb_per_game', 'dreb_per_game', 'trb_per_game', 'ast_per_game', 'stl_per_game', 'blk_per_game', 'tov_per_game', 'pf_per_game', 'pts_per_48', 'fg_per_48', '3p_per_48', 'ft_per_48', 'oreb_per_48', 'dreb_per_48', 'trb_per_48', 'ast_per_48', 'stl_per_48', 'blk_per_48', 'tov_per_48', 'pf_per_48', 'pts_per_100', 'fg_per_100', '3p_per_100', 'ft_per_100', 'oreb_per_100', 'dreb_per_100', 'trb_per_100', 'ast_per_100', 'stl_per_100', 'blk_per_100', 'tov_per_100', 'pf_per_100']
Feature matrix shape: (1340, 19)
```

✓ 4 Correlation Analysis & Redundancy Removal

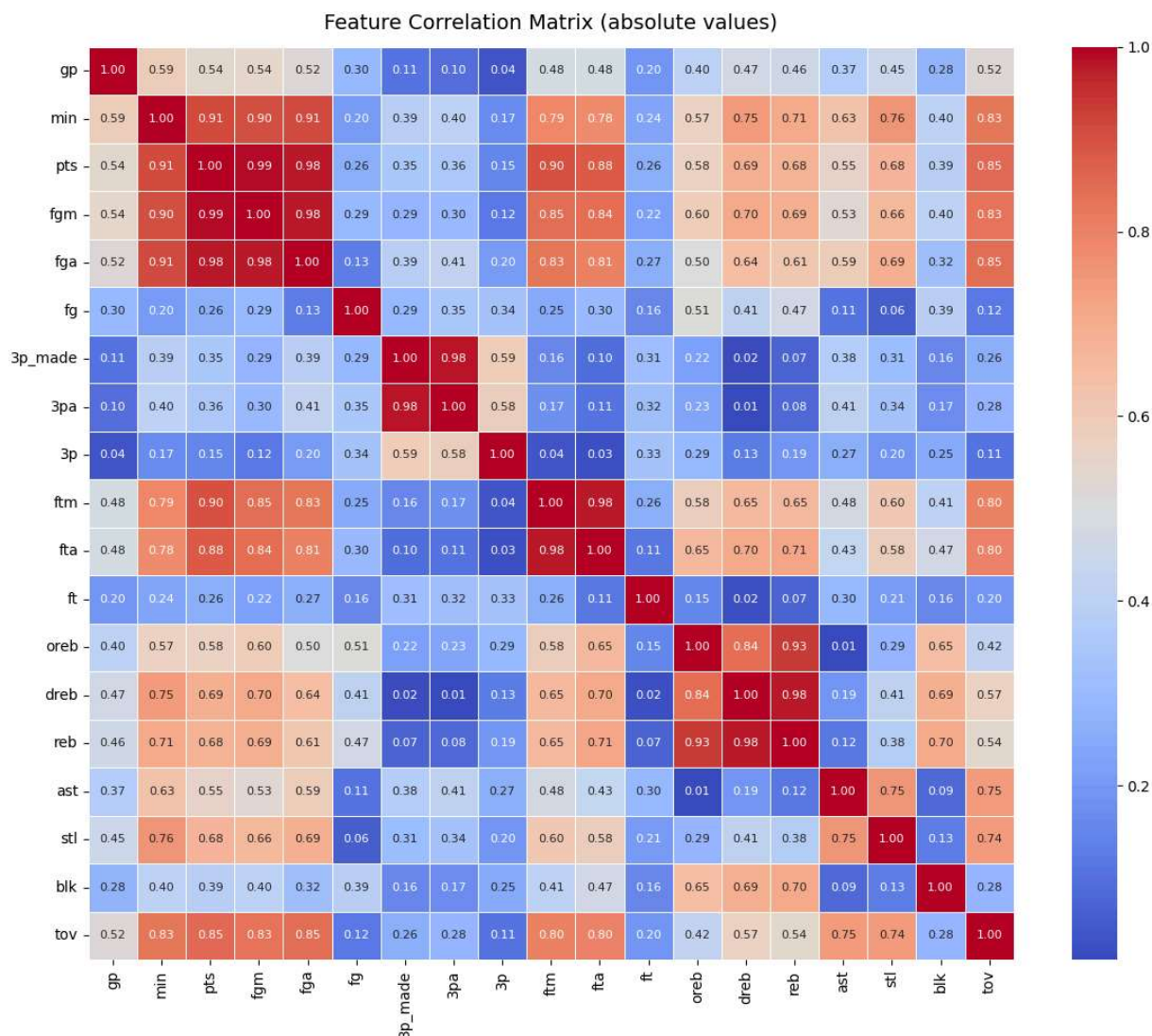
Highly correlated features contribute overlapping information, which can:

- Inflate model variance (multicollinearity)
- Slow convergence for gradient-based models
- Make feature importance scores unreliable

We use a **0.85 absolute correlation threshold** to flag redundant pairs, then keep the feature that is more *interpretable* or *general*.

```
corr_matrix = df.corr().abs()

# Plot full heatmap
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, ax=ax, annot_kws={'size': 8})
ax.set_title('Feature Correlation Matrix (absolute values)', fontsize=14, pad=1)
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=120)
plt.show()
print("Heatmap saved.")
```



Heatmap saved.

```
# Identify pairs exceeding the threshold
THRESHOLD = 0.85
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool), 0.85)

high_corr_pairs = (
    upper_tri.stack()
    .reset_index()
    .rename(columns={'level_0': 'Feature A', 'level_1': 'Feature B', 0: 'Correlation'})
)
```

```

        .query("Correlation > @THRESHOLD")
        .sort_values('Correlation', ascending=False)
    )

    print(f"Feature pairs with |r| > {THRESHOLD}:")
    display(high_corr_pairs.reset_index(drop=True))

```

Feature pairs with $|r| > 0.85$:

	Feature A	Feature B	Correlation	
0	pts	fgm	0.991	
1	3p_made	3pa	0.983	
2	ftm	fta	0.981	
3	fgm	fga	0.980	
4	pts	fga	0.980	
5	dreb	reb	0.978	
6	oreb	reb	0.933	
7	min	pts	0.912	
8	min	fga	0.910	
9	min	fgm	0.903	
10	pts	ftm	0.896	
11	pts	fta	0.881	
12	pts	tov	0.850	

✓ 4.1 Drop Redundant Features

Dropping rationale for each flagged pair:

Dropped	Kept	Reason
fgm	fga	Field goals made is almost perfectly predicted by attempts; attempts capture <i>usage</i> better
ftm	fta	Same logic — free-throw makes $\approx$ makes $\times$ FT% (already captured by ft)
3p_made	3pa	Attempts capture volume; the percentage 3p captures efficiency
fga	pts	We keep pts as the single scoring summary; attempts stay as a volume proxy only — but both f
dreb	reb	Total rebounds (reb) already includes defensive boards
oreb	reb	Same — reb is the aggregate

Note: min correlates strongly with pts / fga / fgm because high-minute players naturally accumulate more stats. We retain min because it is a direct measure of coach trust / role, not just a scoring proxy.

```
# Columns to drop based on correlation analysis
drop_corr = ['fgm', 'ftm', '3p_made', 'oreb', 'dreb']

df = df.drop(columns=drop_corr)
print("Dropped:", drop_corr)
print("Remaining features:", df.columns.tolist())
print("Shape after redundancy removal:", df.shape)
```

```
Dropped: ['fgm', 'ftm', '3p_made', 'oreb', 'dreb']
Remaining features: ['gp', 'min', 'pts', 'fga', 'fg', '3pa', '3p', 'fta', 'ft',
Shape after redundancy removal: (1340, 14)
```

✓ 5 Engineer Composite Features

Raw counting stats are scale-dependent — a player who logs 35 minutes will naturally record more points than one playing 15 minutes. We create rate-based and efficiency metrics that normalise for playing time and usage.

5.1 Points Per Minute (PPM)

Captures scoring *efficiency* independent of playing time.

```
# Guard against division by zero (min = 0 is impossible in this dataset but got
df['ppm'] = (df['pts'] / df['min']).replace(0, np.nan).round(4)

print("PPM — sample statistics:")
print(df['ppm'].describe())
```

```
PPM — sample statistics:
count    1340.000
mean       0.371
std        0.094
min        0.122
25%        0.307
50%        0.363
75%        0.431
max        0.739
Name: ppm, dtype: float64
```

✓ 5.2 Player Efficiency Rating — Simplified (PER_simple)

A lightweight approximation inspired by John Hollinger's PER.

Combines positive contributions (pts, reb, ast, stl, blk) and subtracts turnovers, normalised by minutes:

$$\text{PER_simple} = \frac{\text{pts} + \text{reb} + \text{ast} + \text{stl} + \text{blk} - \text{tov}}{\text{min}}$$

This single number captures multi-dimensional player impact on a per-minute basis.

```
df['per_simple'] = (
    (df['pts'] + df['reb'] + df['ast'] + df['stl'] + df['blk'] - df['tov'])
    / df['min'].replace(0, np.nan)
).round(4)

print("PER_simple - sample statistics:")
print(df['per_simple'].describe())
```

```
PER_simple - sample statistics:
count    1340.000
mean      0.614
std       0.110
min       0.274
25%       0.540
50%       0.609
75%       0.680
max       1.115
Name: per_simple, dtype: float64
```

✓ 5.3 True Shooting Percentage (TS%)

Accounts for the fact that 3-pointers and free throws are more efficient per possession than 2-point attempts:

$$\text{TS\%} = \frac{\text{pts}}{2 \times (\text{fga} + 0.44 \times \text{fta})}$$

```
denominator = 2 * (df['fga'] + 0.44 * df['fta'])
df['ts_pct'] = (df['pts'] / denominator.replace(0, np.nan)).round(4)

print("TS% - sample statistics:")
print(df['ts_pct'].describe())
```

```
TS% - sample statistics:
count    1340.000
mean      0.501
std       0.053
min       0.278
25%       0.470
50%       0.503
75%       0.536
max       0.708
Name: ts_pct, dtype: float64
```

5.4 Assist-to-Turnover Ratio (AST_TOV)

A key decision-making metric — measures how many assists a player creates per turnover committed.

```
df['ast_tov'] = (df['ast'] / df['tov'].replace(0, np.nan)).round(4)

print("AST/TOV — sample statistics:")
print(df['ast_tov'].describe())
```

```
AST/TOV — sample statistics:
count    1340.000
mean       1.241
std        0.723
min        0.000
25%        0.708
50%        1.096
75%        1.628
max        6.000
Name: ast_tov, dtype: float64
```

6 Handle Missing & Infinite Values

Edge cases can arise from divisions (e.g., `tov = 0` → `ast_tov = inf`).

We inspect, then apply a principled imputation strategy:

- **Infinite values** → replace with `NaN` first, then impute
- **NaN in engineered features** → impute with the **column median** (robust to skew, avoids mean-distortion from outliers)
- **NaN in original features** → there are none in this dataset, confirmed below

```
# Replace inf/-inf with NaN
df.replace([np.inf, -np.inf], np.nan, inplace=True)

print("Null counts after feature engineering:")
null_counts = df.isnull().sum()
print(null_counts[null_counts > 0] if null_counts.any() else " No nulls found.")
```

```
Null counts after feature engineering:
No nulls found.
```

```
# Median imputation for engineered columns (no leakage: computed on full train)
engineered_cols = ['ppm', 'per_simple', 'ts_pct', 'ast_tov']
for col in engineered_cols:
    median_val = df[col].median()
    n_filled = df[col].isnull().sum()
```



```

df[col] = df[col].fillna(median_val)
if n_filled > 0:
    print(f" {col}: filled {n_filled} NaN(s) with median={median_val:.4f}")

print("\nNull counts after imputation:")
print(df.isnull().sum().sum(), "total nulls remaining")

```

Null counts after imputation:
0 total nulls remaining

✓ 7 Final Feature Set Overview

```

# Reassemble model-ready dataframe
df_model = df.copy()
df_model['target_5yrs'] = target.values

print("Final dataset shape:", df_model.shape)
print("\nFeature descriptions:")
feature_meta = {
    'gp': 'Games played',
    'min': 'Minutes per game',
    'pts': 'Points per game',
    'fga': 'Field goal attempts per game',
    'fg': 'Field goal percentage',
    '3pa': '3-point attempts per game',
    '3p': '3-point percentage',
    'fta': 'Free throw attempts per game',
    'ft': 'Free throw percentage',
    'reb': 'Total rebounds per game',
    'ast': 'Assists per game',
    'stl': 'Steals per game',
    'blk': 'Blocks per game',
    'tov': 'Turnovers per game',
    'ppm': '[ENGINEERED] Points per minute',
    'per_simple': '[ENGINEERED] Simplified player efficiency rating',
    'ts_pct': '[ENGINEERED] True shooting percentage',
    'ast_tov': '[ENGINEERED] Assist-to-turnover ratio',
}
for feat, desc in feature_meta.items():
    print(f" {feat:<12} {desc}")
print("\n target_5yrs Binary target (1 = played ≥ 5 seasons)")

```

Final dataset shape: (1340, 19)

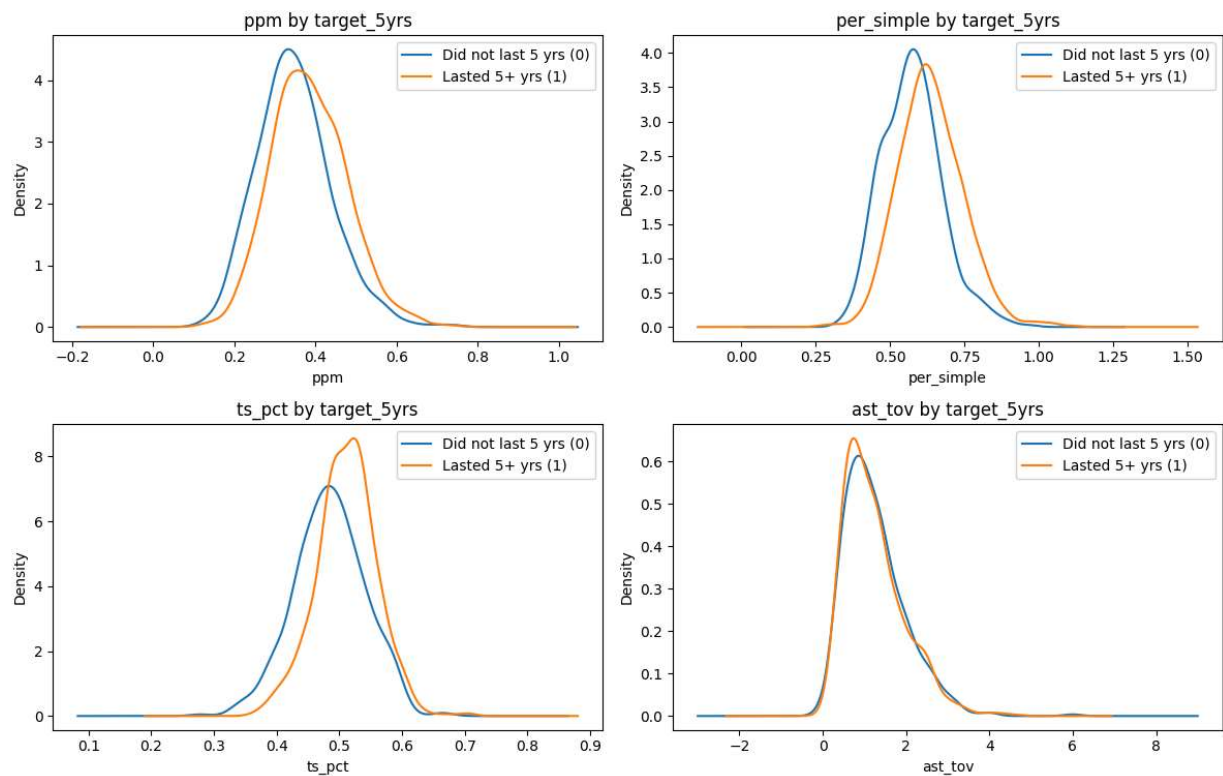
Feature descriptions:

gp	Games played
min	Minutes per game
pts	Points per game
fga	Field goal attempts per game

fg	Field goal percentage
3pa	3-point attempts per game
3p	3-point percentage
fta	Free throw attempts per game
ft	Free throw percentage
reb	Total rebounds per game
ast	Assists per game
stl	Steals per game
blk	Blocks per game
tov	Turnovers per game
ppm	[ENGINEERED] Points per minute
per_simple	[ENGINEERED] Simplified player efficiency rating
ts_pct	[ENGINEERED] True shooting percentage
ast_tov	[ENGINEERED] Assist-to-turnover ratio
target_5yrs	Binary target (1 = played $\geq$ 5 seasons)

```
# Distribution of engineered features vs target
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, col in zip(axes.flatten(), engineered_cols):
    df_model.groupby('target_5yrs')[col].plot.kde(ax=ax, legend=True)
    ax.set_title(f'{col} by target_5yrs')
    ax.set_xlabel(col)
    ax.legend(['Did not last 5 yrs (0)', 'Lasted 5+ yrs (1)'])
plt.suptitle('Engineered Feature Distributions by Target Class', fontsize=13, y
plt.tight_layout()
plt.savefig('engineered_features_dist.png', dpi=120)
plt.show()
```

Engineered Feature Distributions by Target Class



✓ 8 Export Clean Dataset

```
df_model.to_csv('nba_features_clean.csv', index=False)
print("Saved: nba_features_clean.csv")
print("Final shape:", df_model.shape)
df_model.head()
```

```

Saved: nba_features_clean.csv
Final shape: (1340, 19)
-----gp-----min-----pts-----fga-----fg-----3pa-----3p-----fta-----ft-----reb-----ast-----stl
Next steps: 0 36 27.400 7.400 7.600 34.700 2.100 25.000 2.300 69.900 4.100 1.900 0.400

```

1	35	26.900	7.200	6.700	29.600	2.800	23.500	3.400	76.500	2.400	3.700	1.100
---	----	--------	-------	-------	--------	-------	--------	-------	--------	-------	-------	-------

9 Summary of Feature Engineering Decisions